

Table of Contents

Chapter 5: Code Testing and Debugging	3
A Complete Study Guide for Grade 12 Computer Science	3
Before You Begin: A Note to You, the Student	3
5.1 Introduction	3
5.1.1 The Software Life Cycle: Moving from Writing Code to Verifying Code	3
5.1.2 The Mindset of a Detective: Why Code Rarely Works on the First Try	4
5.2 Testing and Debugging Techniques	6
5.2.1 Importance of Testing and Debugging	6
Why Testing Is Essential for Reliable Applications	6
Common Types of Programming Errors and Bugs	7
Unit Testing	11
Introduction to Automated Testing Concepts (Assertions)	11
Introduction to Python's unittest Module	12
Introduction to pytest Framework	15
Writing Test Cases and Edge Case Analysis	17
Using Breakpoints and Watches	20
Setting Breakpoints in IDEs like PyCharm or VS Code	20
Monitoring Variable Values with Watch Expressions	21
The Step-by-Step Debugging Process	22
Exception Handling	23
Understanding Python's try, except, else, and finally Blocks	23
Handling Multiple Exception Types Effectively	26
Raising Custom Exceptions and Maintaining Graceful Failure	26
5.3 Profiling and Optimization	28
Identifying and Fixing Performance Bottlenecks	28
Time Profiling: Measuring Execution Speed	28
Space / Memory Profiling Concepts	31
Refactoring Strategies	32
Chapter Summary	34
Glossary of Key Terms	34
Quick Reference: Choosing the Right Tool	36
Key Principles to Remember	36
Exercise Section	37
Multiple Choice Questions	37

Short Questions	39
Long Questions	39

Chapter 5: Code Testing and Debugging

A Complete Study Guide for Grade 12 Computer Science

"The computer was doing what I told it to do. I just hadn't told it what I meant."

— Every programmer, at least once a week.

Before You Begin: A Note to You, the Student

Bugs are not a sign that you are a bad programmer. They are a sign that you are a programmer. Every software developer — from a first-year student to a 30-year veteran at Google — spends a significant part of every working day chasing errors, reading red text, and asking "why is this not working?"

This chapter is your guide to turning that frustration into a superpower. By the end of it, you will not just fix bugs by accident. You will hunt them systematically, prove your code works with automated tests, and measure exactly how fast your program runs — down to the millisecond.

Let's begin.

5.1 Introduction

5.1.1 The Software Life Cycle: Moving from Writing Code to Verifying Code

The Hook — The Day a Moth Crashed a Computer

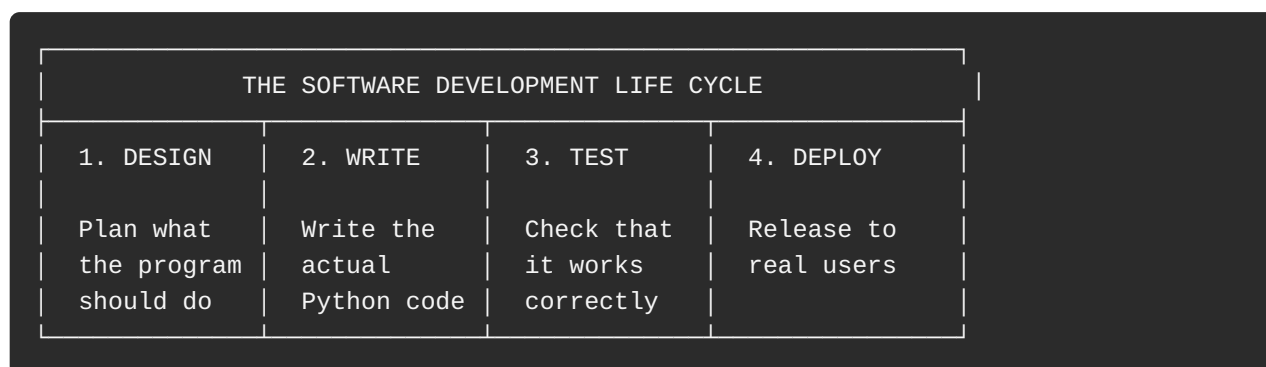
On the 9th of September, 1947, a team of engineers at Harvard University were running tests on a large computing machine called the **Harvard Mark II**. The machine kept producing incorrect results. After hours of investigation, they opened the hardware panels and found the cause: a real, physical moth had flown inside the machine and gotten trapped between the contacts of Relay Number 70, causing it to malfunction.

The lead engineer, **Rear Admiral Grace Hopper**, taped the dead moth into the team's logbook with the note: *"First actual case of bug being found."*

That moth is now preserved at the Smithsonian National Museum of American History. And from that day forward, the word **"bug"** became the universal term for any error inside a program, and **"debugging"** became the process of finding and removing it.

The Explanation

When you write a program, you are moving through what engineers call the **software development life cycle**. Think of it as the stages every piece of software passes through before it reaches real users.



Most beginners focus almost entirely on stage 2 — writing the code. But professional developers know that stages 3 and 4 together often take *more time* than writing the code itself.

Why? Because writing code that *runs* is easy. Writing code that *works correctly, every time, for every possible input, without crashing* is very hard.

Testing is the process of checking that your program produces the correct output for a given input. It is how you prove — not just hope — that your code works.

Debugging is the process of finding and fixing the mistakes when your program does not work correctly.

These two skills together are what separate an amateur programmer from a professional engineer.

Quick Recap: Writing code is only half the job. Testing proves it works. Debugging fixes it when it doesn't. Every professional programmer practices both, every single day.

5.1.2 The Mindset of a Detective: Why Code Rarely Works on the First Try

The Hook — The \$370 Million Rocket That Destroyed Itself

On the 4th of June, 1996, the **Ariane 5 rocket** launched from French Guiana carrying four scientific satellites. Thirty-seven seconds after liftoff, the rocket violently veered off course, broke apart, and exploded. Total cost: approximately **\$370 million USD**, destroyed in 37 seconds.

The cause? A single software bug.

The flight control computer tried to convert a 64-bit floating-point number (a very large decimal number) into a 16-bit integer (a much smaller storage type). The number was too large to fit. The conversion failed, and instead of handling this error gracefully, the program crashed completely. The backup computer ran the same software — and crashed too. With no guidance system, the rocket self-destructed.

The entire disaster could have been prevented by one technique you will learn in this chapter: **exception handling** — writing code that plans for failure and responds gracefully instead of crashing.

The Explanation

Here is a mindset shift that will change how you feel about programming:

A bug in your code is not a personal failure. It is a clue waiting to be found.

Think of yourself as a **detective**. A detective does not get frustrated when a crime is hard to solve. They follow a systematic process: gather evidence, form a hypothesis, test it, and either confirm or rule it out.

Debugging works the same way:

THE DETECTIVE DEBUGGING PROCESS	
Step 1	OBSERVE — What is the wrong output or error message?
Step 2	HYPOTHESIZE — Where in the code could this go wrong?
Step 3	TEST — Set a breakpoint or add a test. Run the code.
Step 4	ANALYZE — Does your hypothesis match what you see?
Step 5	FIX — Change the code and test again.
Step 6	VERIFY — Run the full test suite. Is everything green?

Senior software engineers at companies like Amazon or Microsoft follow this exact process. The only difference between them and a beginner is that they have better tools — and that is exactly what this chapter will give you.

Pause & Think

A student writes a function that is supposed to calculate a student's grade. They run it and see the output **87** instead of the expected **92**. The program did not crash. There was no red error message.

Question: What type of error is this likely to be? Why is this type of error sometimes harder to find than an error that crashes the program?

(Think about this before reading Section 5.2. The answer will become clear.)

Quick Recap: Approach bugs like a detective — systematically, with tools, not with panic. Good programmers are not people who write bug-free code on the first try. They are people who know how to find and fix bugs efficiently.

5.2 Testing and Debugging Techniques

5.2.1 Importance of Testing and Debugging

The Hook — When No Testing Kills People

Between 1985 and 1987, a radiation therapy machine called the **Therac-25** was used in hospitals across the United States and Canada. It was supposed to deliver precise, controlled doses of radiation to treat cancer patients.

Due to a **race condition bug** (a timing error where two parts of the program ran at the wrong moment) and the removal of hardware safety locks that previous versions had relied on, the machine sometimes delivered radiation doses **100 times higher than intended**.

At least six patients received lethal or near-lethal radiation overdoses. Three of them died directly as a result.

The software had never been properly tested under specific rapid-input conditions. The engineers assumed the hardware would catch errors that the software missed — but the hardware safety locks had been removed.

This is the most sobering example in all of computer science history of why **testing is not optional**. In safety-critical systems, the absence of testing is the absence of conscience.

Why Testing Is Essential for Reliable Applications

The Explanation

Imagine you are building a bridge. You do not build the bridge and then wait for a bus full of passengers to drive across it to find out if it holds. You test every beam, every bolt, every weld — first in isolation, then together.

Software is the same. You test each function individually first. Then you test how the functions work together. Then you test the whole system.

Here is what testing accomplishes:

Testing Goal	What It Does
Verification	Confirms the function produces the correct output
Edge Case Coverage	Tests unusual or extreme inputs (0, negative, very large)
Regression Prevention	Ensures new code does not break old, working features
Documentation	Test cases serve as living examples of how a function should behave
Confidence	Allows you to change code boldly, knowing tests will catch mistakes

Without testing, every change you make to a large program is a gamble. With testing, every change you make is an experiment with immediate feedback.

Common Types of Programming Errors and Bugs

The Explanation — The Three Villains of Programming

Every bug you will ever encounter in Python falls into one of three categories. Understanding the category immediately tells you *where to look* and *which tool to use*.

Everyday Analogy — The Restaurant Menu

Imagine writing a restaurant menu:

- **Syntax Error:** You write *"Grilled Chicken"* instead of *"Grilled Chicken."* The printer refuses to print it because the word is misspelled and the system cannot recognize it.
 - **Runtime Error:** The menu looks fine. A customer orders Grilled Chicken. But when the chef goes to make it, they discover there is no chicken in the kitchen. The process crashes mid-execution.
 - **Logic Error:** The chef cooks the chicken perfectly. But someone accidentally swapped the salt and sugar containers in the morning. The dish is served — it just tastes terrible. No alarm went off. The output just wasn't what anyone expected.
-

Villain #1: Syntax Errors

A **syntax error** occurs when your code breaks the grammar rules of Python. Python cannot even begin to run the program. It stops immediately and shows you an error.

Definition: A *syntax error* is a mistake in the structure or spelling of your code that Python cannot parse (read and understand).

```
# BUGGY CODE – Syntax Error Example
if x > 5
    print("x is greater")  # Missing colon after the if condition
```

```
File "program.py", line 1
    if x > 5
        ^
SyntaxError: expected ':'
```

The caret symbol (**^**) in Python's error message is your friend. It points directly to the problem.

How to fix it: Read the error message. Look at the line number. Check your colons, brackets, parentheses, and quotation marks.

Villain #2: Runtime Errors (Exceptions)

A **runtime error** occurs while the program is *running*. The syntax was correct — Python started the program — but something went wrong during execution.

Definition: A *runtime error* (also called an *exception*) is an error that occurs during program execution, causing it to stop unexpectedly.

```
# BUGGY CODE – Runtime Error Example
def divide(a, b):
    return a / b

result = divide(10, 0)  # You cannot divide by zero!
print(result)
```

```
Traceback (most recent call last):
  File "program.py", line 4, in <module>
    result = divide(10, 0)
  File "program.py", line 2, in divide
    return a / b
ZeroDivisionError: division by zero
```

This output is called a **stack trace**. It is not scary — it is a map of exactly where your program was when it crashed.

Definition: A *stack trace* is a report that shows the sequence of function calls that led to the error, listed from most recent (bottom) to earliest (top).

How to read a stack trace:

1. Always read from the **bottom up**.
2. The bottom line tells you the *type* of error (`ZeroDivisionError`) and a plain description.
3. The lines above show the *chain of function calls* that led there.
4. Find *your* file name (not Python's built-in files) and look at that line first.

Common Python Runtime Errors:

Exception Name	When It Happens
<code>ZeroDivisionError</code>	Dividing any number by zero
<code>ValueError</code>	Passing the wrong type of value (e.g., <code>int("hello")</code>)
<code>TypeError</code>	Performing an operation on incompatible types
<code>IndexError</code>	Accessing a list position that does not exist
<code>KeyError</code>	Accessing a dictionary key that does not exist
<code>FileNotFoundError</code>	Opening a file that does not exist
<code>NameError</code>	Using a variable name that was never defined

Villain #3: Logic Errors

A **logic error** is the sneakiest villain. The program runs perfectly. No red error messages appear. But the output is wrong.

Definition: A *logic error* is a mistake in the reasoning or algorithm of the code. The program runs without crashing but produces incorrect results.

```
# BUGGY CODE – Logic Error Example
# Supposed to check if a student PASSED (score >= 50)
def check_result(score):
    if score > 50:          # BUG: Should be >= 50, not > 50
        return "Pass"
    else:
        return "Fail"

print(check_result(50))  # Outputs: "Fail" – WRONG! 50 should be a Pass!
```

No error message. No crash. Just a wrong answer.

How to find logic errors: This is exactly what **unit testing** and **breakpoint debugging** are designed to catch. You will learn both in this chapter.

ERROR TYPE COMPARISON TABLE		
Error Type	When Detected?	Best Tool to Find It
Syntax Error	Before running	Read the error message & line
Runtime Error	While running	Stack trace + Exception Handler
Logic Error	After running (output is wrong)	Unit Tests + Breakpoints

Pause & Think

Look at this code:

```
def calculate_average(numbers):
    total = 0
    for num in numbers:
        total = total + num
    return total / len(numbers)

scores = [85, 90, 78, 92, 88]
print(calculate_average(scores))      # Works fine
print(calculate_average([]))          # What happens here? Why?
```

Question 1: What type of error will the second call produce? Write down the exact exception name you expect before running it.

Question 2: What change to the function would prevent this error from crashing the program?

Quick Recap: Every programming error is one of three types — syntax (grammar broken), runtime (crash during execution), or logic (wrong output, no crash). Knowing the type tells you exactly which tool and approach to use.

Unit Testing

Introduction to Automated Testing Concepts (Assertions)

The Hook — The Car Factory

Imagine a car factory assembling hundreds of cars per day. Before any car is assembled, every single part is tested on its own:

- The brakes are tested alone on a special workbench.
- The headlights are tested alone in a dark room.
- The engine's spark plugs are tested alone with a voltage tester.

If a brake pad is defective, it is caught *before* it goes into a car. The cost of replacing one brake pad is tiny. The cost of recalling ten thousand cars because all of them have defective brakes is catastrophic.

Unit testing applies this exact principle to software. You test each function — each individual "part" — in isolation, before connecting them into a full program.

The Explanation

Definition: *Unit testing* is the practice of writing small, automated test programs that check whether individual functions or "units" of your code produce the correct output for specific inputs.

The core tool of unit testing is the **assertion**.

Definition: An *assertion* is a statement that checks whether a condition is true. If it is true, the test passes silently. If it is false, the test immediately fails and shows an error.

```
# Manual assertion example (simplest form)
result = 2 + 3
assert result == 5, "Expected 5 but got something else"
# If result were 6, Python would raise an AssertionError here
```

The word `assert` in Python means: *"I am asserting — declaring as true — that this condition holds. If it does not, something is wrong."*

Introduction to Python's `unittest` Module

The Explanation

Python includes a built-in testing module called `unittest`. It gives you a structured, organized way to group your tests into **test cases** and **test suites**.

Definition: A test case is a class that contains one or more test methods, each checking a specific behavior of your code.

Let's build a complete example. Suppose you have written this function:

```
# calculator.py ← Save this file separately

def add(a, b):
    """Returns the sum of two numbers."""
    return a + b

def subtract(a, b):
    """Returns the difference of two numbers."""
    return a - b

def multiply(a, b):
    """Returns the product of two numbers."""
    return a * b

def divide(a, b):
    """Returns the result of dividing a by b.
    Raises ZeroDivisionError if b is zero."""
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero")
    return a / b
```

Now here is the complete `unittest` test suite for this calculator:

```

# test_calculator_unittest.py ← Save this as a separate file

import unittest
from calculator import add, subtract, multiply, divide

class TestCalculator(unittest.TestCase):

    # — TESTS FOR add() —————
    def test_add_positive_numbers(self):
        """add(2, 3) should return 5"""
        result = add(2, 3)
        self.assertEqual(result, 5)

    def test_add_negative_numbers(self):
        """add(-4, -6) should return -10"""
        self.assertEqual(add(-4, -6), -10)

    def test_add_zero(self):
        """Adding zero should return the other number unchanged"""
        self.assertEqual(add(7, 0), 7)

    # — TESTS FOR subtract() —————
    def test_subtract_positive_numbers(self):
        self.assertEqual(subtract(10, 4), 6)

    def test_subtract_produces_negative(self):
        """Subtracting larger from smaller should give a negative result"""
        self.assertEqual(subtract(3, 10), -7)

    # — TESTS FOR multiply() —————
    def test_multiply_basic(self):
        self.assertEqual(multiply(3, 4), 12)

    def test_multiply_by_zero(self):
        """Multiplying anything by zero should return 0"""
        self.assertEqual(multiply(99, 0), 0)

    def test_multiply_negatives(self):
        """Negative × Negative = Positive"""
        self.assertEqual(multiply(-3, -4), 12)

    # — TESTS FOR divide() —————
    def test_divide_basic(self):
        self.assertEqual(divide(10, 2), 5.0)

    def test_divide_by_zero_raises_error(self):
        """divide(x, 0) must raise ZeroDivisionError, not return None"""
        with self.assertRaises(ZeroDivisionError):
            divide(10, 0)

    def test_divide_produces_decimal(self):
        self.assertAlmostEqual(divide(1, 3), 0.333, places=3)

```

```
# This block runs the test suite when you execute this file directly
if __name__ == "__main__":
    unittest.main()
```

How to run it:

```
python test_calculator_unittest.py
```

Sample Output when ALL tests pass:

```
.....
-----
Ran 10 tests in 0.003s

OK
```

Each **.** (dot) represents one passing test. Ten dots = ten tests passed.

Sample Output when ONE test FAILS:

```
.....F....
=====
FAIL: test_add_positive_numbers (__main__.TestCalculator)
-----
Traceback (most recent call last):
  File "test_calculator_unittest.py", line 12, in test_add_positive_numbers
    self.assertEqual(result, 5)
AssertionError: 6 != 5
-----
Ran 10 tests in 0.004s

FAILED (failures=1)
```

Important Mindset: See that **FAIL** in red? That is not a punishment. That is a **gift**. It means your test caught a bug before a real user did. A failing test is a successful alarm system.

Key **unittest** assertion methods:

Method	What it checks
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>x</code> is <code>True</code>
<code>assertFalse(x)</code>	<code>x</code> is <code>False</code>
<code>assertRaises(Error, func, args)</code>	<code>func(args)</code> raises the specified Error
<code>assertAlmostEqual(a, b, places=n)</code>	<code>a ≈ b</code> up to <code>n</code> decimal places
<code>assertIsNone(x)</code>	<code>x</code> is <code>None</code>

Introduction to `pytest` Framework

The Explanation

`pytest` is an external testing library that makes writing tests even simpler. It does not require you to create a class. Your test functions stand on their own. `pytest` also gives you clearer, more colorful output.

Definition: `pytest` is a Python testing framework that automatically discovers and runs any function whose name starts with `test_`.

Install it once:

```
pip install pytest
```

Here is the same calculator test suite rewritten in `pytest` style:

```

# test_calculator_pytest.py

import pytest
from calculator import add, subtract, multiply, divide

# — TESTS FOR add() —————
def test_add_positive_numbers():
    assert add(2, 3) == 5

def test_add_negative_numbers():
    assert add(-4, -6) == -10

def test_add_zero():
    assert add(7, 0) == 7

# — TESTS FOR subtract() —————
def test_subtract_basic():
    assert subtract(10, 4) == 6

def test_subtract_produces_negative():
    assert subtract(3, 10) == -7

# — TESTS FOR multiply() —————
def test_multiply_basic():
    assert multiply(3, 4) == 12

def test_multiply_by_zero():
    assert multiply(99, 0) == 0

# — TESTS FOR divide() —————
def test_divide_basic():
    assert divide(10, 2) == 5.0

def test_divide_by_zero_raises_error():
    with pytest.raises(ZeroDivisionError):
        divide(10, 0)

def test_divide_decimal_result():
    assert abs(divide(1, 3) - 0.333) < 0.001

```

How to run it:

```
pytest test_calculator_pytest.py -v
```

Sample Output:

```

===== test session starts =====
collected 9 items

test_calculator_pytest.py::test_add_positive_numbers PASSED [ 11%]
test_calculator_pytest.py::test_add_negative_numbers PASSED [ 22%]
test_calculator_pytest.py::test_add_zero PASSED [ 33%]
test_calculator_pytest.py::test_subtract_basic PASSED [ 44%]
test_calculator_pytest.py::test_subtract_produces_negative PASSED [ 55%]
test_calculator_pytest.py::test_multiply_basic PASSED [ 66%]
test_calculator_pytest.py::test_multiply_by_zero PASSED [ 77%]
test_calculator_pytest.py::test_divide_basic PASSED [ 88%]
test_calculator_pytest.py::test_divide_by_zero_raises_error PASSED [100%]

===== 9 passed in 0.12s =====

```

Writing Test Cases and Edge Case Analysis

The Explanation

Writing a test for the normal case ($2 + 3 = 5$) is the easy part. The real skill is identifying **edge cases** — the unusual, extreme, or unexpected inputs that most developers forget to test.

Definition: An *edge case* is an input value at the extreme boundary of what is valid — like zero, negative numbers, empty strings, or the maximum possible value.

A Systematic Approach to Edge Case Analysis:

EDGE CASE CHECKLIST FOR ANY FUNCTION	
1	What is the SMALLEST valid input? (e.g., 0, empty list)
2	What is the LARGEST valid input? (e.g., 1,000,000)
3	What happens with NEGATIVE values?
4	What happens with WRONG TYPES? (e.g., string instead of int)
5	What is the BOUNDARY condition? (e.g., exactly 50 in ≥ 50)

Full Example — Testing a Discount Calculator:

```
# discount.py

def calculate_discount(price, discount_percent):
    """
    Returns the final price after applying a discount.
    Raises ValueError if price is negative or discount is not between 0 and 100.
    """
    if price < 0:
        raise ValueError("Price cannot be negative")
    if discount_percent < 0 or discount_percent > 100:
        raise ValueError("Discount must be between 0 and 100")

    discount_amount = price * (discount_percent / 100)
    final_price = price - discount_amount
    return round(final_price, 2)
```

```

# test_discount.py

import pytest
from discount import calculate_discount

# — NORMAL CASES —
def test_standard_discount():
    assert calculate_discount(100, 20) == 80.0

def test_large_discount():
    assert calculate_discount(200, 75) == 50.0

# — EDGE CASES — BOUNDARY CONDITIONS —
def test_zero_discount():
    assert calculate_discount(100, 0) == 100.0

def test_full_discount():
    assert calculate_discount(100, 100) == 0.0

def test_zero_price():
    assert calculate_discount(0, 50) == 0.0

# — EDGE CASES — INVALID INPUTS —
def test_negative_price_raises_error():
    with pytest.raises(ValueError):
        calculate_discount(-50, 10)

def test_discount_above_100_raises_error():
    with pytest.raises(ValueError):
        calculate_discount(100, 110)

def test_negative_discount_raises_error():
    with pytest.raises(ValueError):
        calculate_discount(100, -5)

# — EDGE CASES — DECIMAL RESULTS —
def test_decimal_result_rounded_correctly():
    assert calculate_discount(99.99, 33) == 66.99

```

Grab a Partner — Test Writing Challenge

Partner A: Write a `is_leap_year(year)` function.

(A year is a leap year if: divisible by 4, BUT NOT by 100, UNLESS also divisible by 400.)

Partner B: Write at least 6 test cases for `is_leap_year()` before seeing Partner A's implementation. Include:

- A normal leap year (e.g., 2024)
- A normal non-leap year (e.g., 2023)

- A year divisible by 100 but NOT 400 (e.g., 1900 — should be False)
- A year divisible by 400 (e.g., 2000 — should be True)
- Year 0 and a negative year

Then compare: Do Partner A's results match Partner B's expected outputs?

Quick Recap: Unit tests are automated checks that prove each function works correctly. Write tests for normal cases, edge cases, boundary conditions, and invalid inputs. A failing test is not a problem — it is a discovery.

Using Breakpoints and Watches

Setting Breakpoints in IDEs like PyCharm or VS Code

The Hook — The Pause Button

Imagine watching a thriller movie and you suddenly want to check whether the key in the character's pocket matches the lock on the door they're approaching. You hit **Pause**. You walk up to the screen. You look closely at the key's shape. You check the lock's shape. Then you press **Play** again.

A **breakpoint** is the exact equivalent of hitting Pause on your running program. The program freezes at the exact line you choose. You can then look at every variable's value, inspect every data structure, and understand the program's exact state at that moment.

This is infinitely more powerful than `print()` statements, because:

- You do not have to write and then delete dozens of `print()` calls.
 - You can see **all** variables at once, not just the ones you remembered to print.
 - You can jump forward and backward through the code's execution.
 - You can change variable values live to test hypotheses.
-

The Explanation

Definition: A *breakpoint* is a marker that you place on a specific line of code in your IDE. When the program reaches that line during execution, it pauses immediately — before executing that line — and hands control to you.

Setting a Breakpoint in VS Code:

```
VS CODE EDITOR – calculator.py
```

Line	Code
1	def calculate_total(items):
2	total = 0
• 3	for item in items: ← BREAKPOINT SET HERE
4	total += item["price"]
5	return total

How to set a breakpoint in VS Code:

1. Open your Python file.
2. Click in the **left margin** (the empty space next to the line number). A red dot appears.
3. Press **F5** (or go to Run → Start Debugging).
4. The program runs until it hits your breakpoint — then it pauses.

How to set a breakpoint in PyCharm:

1. Open your Python file.
2. Click the **grey area** to the right of the line number. A red circle appears.
3. Right-click the green "Run" button and choose "Debug," OR press **Shift + F9**.
4. The program runs until it hits your breakpoint — then it pauses.

Monitoring Variable Values with Watch Expressions

The Explanation

Definition: A *watch expression* is a formula or variable name that you add to the debugger's Watch panel. The debugger continuously updates its value as you step through the code.

DEBUG PANELS – PAUSED AT LINE 4		
VARIABLES	WATCH EXPRESSIONS	
total = 0 item = {"name": "Pen", "price": 25}	Expression	Value
	total	0
	item["price"]	25
	len(items)	3

How to add a Watch Expression in VS Code:

1. While paused at a breakpoint, go to the **WATCH** panel in the debug view.
 2. Click the **+** icon.
 3. Type any variable name or Python expression.
 4. The watch panel updates its value every time you step to the next line.
-

The Step-by-Step Debugging Process

The Explanation

DEBUGGER NAVIGATION BUTTONS	
F10 (Step Over)	Execute current line, pause at NEXT line. Does NOT go inside called functions.
F11 (Step Into)	If current line calls a function, ENTER that function.
Shift+F11 (Step Out)	Finish current function, return to the calling line.
F5 (Resume)	Continue until next breakpoint or program ends.

The Practical Walkthrough — Finding an Off-by-One Bug

```
# buggy_search.py

def find_first_negative(numbers):
    """Returns the INDEX of the first negative number. Returns -1 if none
    found."""
    for i in range(1, len(numbers)): # BUG: should be range(0, ...)
        if numbers[i] < 0:
            return i
    return -1

data2 = [-5, 10, 20]
result2 = find_first_negative(data2)
print("First negative at index:", result2)
# Expected: 0   Actual: -1   (WRONG! Bug skips index 0)
```

Step-by-Step Debugger Trace for **data2 = [-5, 10, 20]**:

Step	Line Executing	Variable State & Notes
1	<code>find_first_negative([-5, 10, 20])</code>	<code>numbers = [-5, 10, 20]</code>
2	<code>for i in range(1, len(numbers)):</code>	<code>range(1, 3) = [1, 2]</code> <code>i starts at 1, NOT 0 ← PROBLEM!</code>
3	<code>if numbers[i] < 0:</code>	<code>i=1, numbers[1]=10, 10<0 → FALSE</code>
4	(next iteration)	<code>i=2, numbers[2]=20, 20<0 → FALSE</code>
5	<code>return -1</code>	Never checked index 0! WRONG!

ROOT CAUSE: `range(1, len(numbers))` skips index 0.
 FIX: Change to `range(len(numbers))`

The Fixed Code:

```
def find_first_negative(numbers):
    for i in range(len(numbers)):    # Now starts at index 0
        if numbers[i] < 0:
            return i
    return -1
```

Pause & Think — The `print()` vs. Debugger Question

A student says: "I just put `print(i)` everywhere. I don't need a debugger."

Question: List two specific situations where using a debugger with breakpoints and watch expressions is significantly more efficient than using `print()` statements.

Quick Recap: Breakpoints pause your program mid-execution. Watch expressions show you variable values in real time. Step Over, Step Into, Step Out, and Resume let you navigate through the code with surgical precision.

Exception Handling

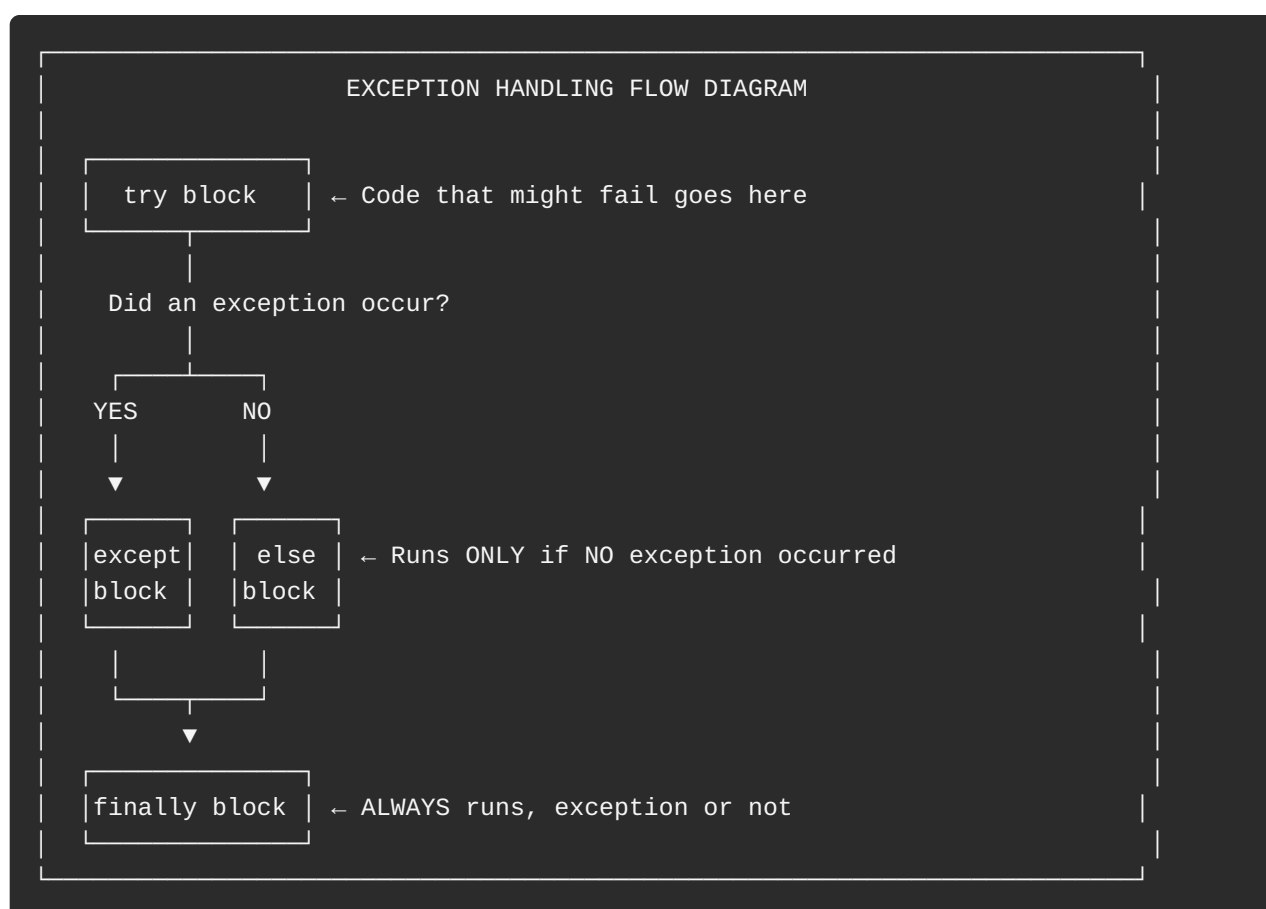
Understanding Python's `try`, `except`, `else`, and `finally` Blocks

The Hook — The Ariane 5 Rocket, Revisited

We opened this chapter with the Ariane 5 disaster. Now you understand enough to prevent it. The fatal error was an unhandled integer overflow exception. The fix would have been a `try-except` block — just a few lines of code saying: *"If this conversion fails, here is what to do instead of crashing."* A \$370 million rocket, destroyed because those few lines were not written.

The Explanation

Definition: *Exception handling* is the process of writing code that anticipates potential runtime errors and responds to them gracefully — instead of letting the program crash.



Complete Example — User Input Handler:

```
def get_student_score():
    """Asks for a score between 0 and 100. Handles invalid input gracefully."""
    print("Enter student score (0-100):")

    try:
        raw_input = input("Score: ")
        score = int(raw_input)          # Could raise ValueError

        if score < 0 or score > 100:
            raise ValueError(f"Score {score} is out of valid range 0-100")

    except ValueError as error:
        print(f"Invalid input: {error}")
        print("Please enter a whole number between 0 and 100.")
        return None

    else:
        # Runs ONLY if no exception occurred
        print(f"Score {score} accepted successfully.")
        return score

    finally:
        # ALWAYS runs
        print("Input attempt completed.")

result = get_student_score()
if result is not None:
    print(f"Final score: {result}")
```

Execution Trace — User types **"hello"**:

```
Output:
Enter student score (0-100):
Score: hello
Invalid input: invalid literal for int() with base 10: 'hello'
Please enter a whole number between 0 and 100.
Input attempt completed.
```

Execution Trace — User types **75**:

```
Output:
Enter student score (0-100):
Score: 75
Score 75 accepted successfully.
Input attempt completed.
```

Handling Multiple Exception Types Effectively

The Explanation

A single program may encounter different types of exceptions. Python allows catching multiple exception types, each with a specific response.

```
# multiple_exceptions.py

def process_student_record(student_data, index):
    """Retrieves and processes a student's grade. Handles multiple exception
    types."""
    try:
        student = student_data[index]          # Could raise IndexError
        grade = student["grade"]                # Could raise KeyError
        grade_number = float(grade)             # Could raise ValueError
        max_grade = student["max_grade"]
        percentage = (grade_number / max_grade) * 100 # Could raise
        ZeroDivisionError
        return round(percentage, 2)

    except IndexError:
        print(f"Error: No student found at position {index}.")
        return None

    except KeyError as missing_key:
        print(f"Error: Missing field: {missing_key}")
        return None

    except ValueError as e:
        print(f"Error: Invalid grade value. Details: {e}")
        return None

    except ZeroDivisionError:
        print("Error: max_grade cannot be zero.")
        return None

    finally:
        print(f"Finished processing attempt for index {index}.\n")
```

Raising Custom Exceptions and Maintaining Graceful Failure

The Explanation

Definition: A *custom exception* is a new exception class that you define yourself, inheriting from Python's built-in `Exception` class. Custom exceptions make your error messages precise and domain-specific.

```
# custom_exceptions.py

class InvalidAgeError(Exception):
    """Raised when a student's age is outside the valid range."""
    pass

class DuplicateStudentError(Exception):
    """Raised when trying to add a student who already exists."""
    pass

class GradeOutOfRangeError(Exception):
    """Raised when a grade value falls outside 0-100."""
    pass

class StudentRegistry:

    def __init__(self):
        self.students = {}

    def add_student(self, name, age):
        if age < 15 or age > 25:
            raise InvalidAgeError(f"Age {age} is invalid. Expected: 15-25.")
        if name in self.students:
            raise DuplicateStudentError(f"Student '{name}' already exists.")
        self.students[name] = {"age": age, "grades": []}
        print(f"Student '{name}' added successfully.")

    def add_grade(self, name, grade):
        if grade < 0 or grade > 100:
            raise GradeOutOfRangeError(f"Grade {grade} is invalid. Must be 0-100.")
        self.students[name]["grades"].append(grade)
        print(f"Grade {grade} added for '{name}'.")
```

Pause & Think — The Ariane 5 Prevention

The Ariane 5 bug crashed when converting a large float to a 16-bit integer (max value: 32,767).

Write a `try-except` block in Python that safely attempts this conversion and prints a meaningful error message if the value exceeds 32,767 — instead of crashing.

Quick Recap: Exception handling separates code that might fail (`try`) from responses to failure (`except`), success actions (`else`), and guaranteed cleanup (`finally`). Custom exceptions make your error messages precise and informative.

5.3 Profiling and Optimization

Identifying and Fixing Performance Bottlenecks

The Hook — The Race Car Pit Stop

Imagine you are the performance engineer for a Formula 1 race team. Your driver is finishing each lap 8 seconds slower than the competition. You use timing data to measure every part of the race: acceleration out of corners (2.1 seconds lost), top speed on straights (0.1 seconds lost), tire change in the pit stop (5.8 seconds lost).

The data is clear: **fix the pit stop first**. That is where almost all your lost time is hiding. Optimizing the acceleration — which loses only 2.1 seconds — before fixing the 5.8-second pit stop would be irrational.

Profiling in programming is exactly this: measuring where your program spends its time, so you know which part to optimize first.

The Explanation

Definition: *Profiling* is the process of measuring a program's execution time and resource usage — broken down by function — to find which parts are slow or memory-heavy.

Definition: A *performance bottleneck* is any part of your code that takes disproportionately longer than other parts, slowing down the entire program.

The key insight is: **never optimize until you have measured**. Programmers frequently waste hours optimizing a function that runs in 0.001 seconds, while ignoring a loop that runs in 9 seconds.

Time Profiling: Measuring Execution Speed

The `time` Module — Simple Timing

```

import time

def slow_search(data, target):
    """Linear search – checks every element one by one."""
    for i, item in enumerate(data):
        if item == target:
            return i
    return -1

def fast_search(data, target):
    """Binary search – halves the search space each step."""
    left, right = 0, len(data) - 1
    while left <= right:
        mid = (left + right) // 2
        if data[mid] == target:
            return mid
        elif data[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

data = list(range(0, 1_000_000))
target = 876_543

start = time.time()
for _ in range(100):
    result = slow_search(data, target)
end = time.time()
slow_time = end - start
print(f"Linear Search – 100 runs: {slow_time:.4f} seconds")

start = time.time()
for _ in range(100):
    result = fast_search(data, target)
end = time.time()
fast_time = end - start
print(f"Binary Search – 100 runs: {fast_time:.4f} seconds")
print(f"Binary search is approximately {slow_time / fast_time:.1f}x faster!")

```

Sample Output:

```

Linear Search – 100 runs: 8.4231 seconds
Binary Search – 100 runs: 0.0003 seconds
Binary search is approximately 28077x faster!

```

The **cProfile** Module — Detailed Function-Level Profiling

```

import cProfile
import timeit

def slow_sum(numbers):
    """Manually adds numbers using a loop."""
    total = 0
    for num in numbers:
        total = total + num
    return total

def fast_sum(numbers):
    """Uses Python's built-in sum() implemented in optimized C code."""
    return sum(numbers)

def process_data(numbers):
    slow_result = slow_sum(numbers)
    fast_result = fast_sum(numbers)
    return slow_result, fast_result

def main():
    numbers = list(range(1, 1_000_001))
    slow_result, fast_result = process_data(numbers)
    print(f"Slow sum result: {slow_result}")
    print(f"Fast sum result: {fast_result}")

    slow_time = timeit.timeit(lambda: slow_sum(numbers), number=10)
    fast_time = timeit.timeit(lambda: fast_sum(numbers), number=10)

    print(f"\nTime Comparison (10 runs each):")
    print(f"  slow_sum() : {slow_time:.4f} seconds")
    print(f"  fast_sum() : {fast_time:.4f} seconds")
    print(f"  Speed difference: {slow_time / fast_time:.1f}x")

print("=== cProfile Report ===")
cProfile.run("main()", sort="tottime")

```

Sample cProfile Output:

```

      8 function calls in 0.520 seconds

Ordered by: internal time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.420    0.420    0.420    0.420 program.py:5(slow_sum)
      1   0.080    0.080    0.520    0.520 program.py:20(main)
      1   0.020    0.020    0.020    0.020 {built-in method builtins.sum}

```

How to read the cProfile output:

cPROFILE OUTPUT — COLUMN GUIDE	
ncalls	How many times this function was called
totttime	Total time spent INSIDE this function (not counting sub-calls)
percall	totttime ÷ ncalls (average time per call)
cumtime	Total time INCLUDING all functions this function called
filename	Which file and line number this function is defined at

`slow_sum` took `0.420` seconds — 80% of the total runtime. This is your bottleneck.

Space / Memory Profiling Concepts

The Explanation

Definition: *Memory profiling* is the process of measuring how much RAM your program uses — broken down by object and data structure — to identify parts that allocate more memory than necessary.

Comparing Memory Usage — List vs. Generator:

```
# Option A: List — creates ALL 1 million numbers in RAM at once
def sum_with_list():
    numbers = list(range(1_000_000))    # ~8MB of RAM
    return sum(numbers)

# Option B: Generator — creates numbers ONE AT A TIME
def sum_with_generator():
    numbers = range(1_000_000)          # ~48 BYTES of RAM
    return sum(numbers)

# Both produce the same answer: 499999500000
# But the generator uses ~166,000x less memory!
```

Concept	What It Means
Memory Leak	Memory allocated but never freed — RAM usage grows endlessly
Peak Memory	Maximum RAM used at any single moment
Memory per Line	How much RAM each line of code allocates

Refactoring Strategies

The Explanation

Definition: *Refactoring* is rewriting code to be faster, more readable, or more memory-efficient — without changing what the code does (the output stays the same).

Strategy 1: Avoid Nested Loops When Possible

```
# BAD —  $O(n^2)$  — 10,000 items = 100,000,000 operations
def find_duplicates_slow(data):
    duplicates = []
    for i in range(len(data)):
        for j in range(i + 1, len(data)):
            if data[i] == data[j]:
                duplicates.append(data[i])
    return duplicates

# GOOD —  $O(n)$  — 10,000 items = ~10,000 operations
def find_duplicates_fast(data):
    seen = set()
    duplicates = set()
    for item in data:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)
```

Strategy 2: Choose the Right Data Structure

DATA STRUCTURE PERFORMANCE COMPARISON		
Operation	List	Set / Dictionary
Check membership	$O(n)$ — SLOW	$O(1)$ — FAST (instant lookup)
Add an item	$O(1)$ at end	$O(1)$ average
Access by index	$O(1)$ — FAST	Not available (unordered)

Strategy 3: Use Built-in Python Functions

Python's built-in functions (`sum()`, `max()`, `min()`, `sorted()`) are implemented in C — a compiled, low-level language — and are significantly faster than equivalent Python loops.

The Professional Optimization Cycle:

THE PROFESSIONAL OPTIMIZATION CYCLE

- 1 | MEASURE FIRST: Run cProfile. Find the slowest function.
- 2 | IDENTIFY BOTTLENECK: Which function has the most tottime?
- 3 | UNDERSTAND THE CAUSE: Nested loop? Wrong data structure?
- 4 | REFACTOR: Apply the appropriate optimization strategy.
- 5 | MEASURE AGAIN: Run cProfile on the new version.
- 6 | RUN TESTS: Confirm the output is still correct.
- 7 | REPEAT: Find the next bottleneck and repeat.

Pause & Think — The Optimization Trap

A program takes 10 seconds. cProfile shows:

ncalls	tottime	function
1	9.500	download_file()
1	0.480	sort_data()
1	0.015	calculate_stats()
1	0.005	format_output()

Question 1: If you optimize `sort_data()` from 0.480 to 0.020 seconds, what is the new total runtime?

Question 2: Is this the right optimization to do first? Why or why not?

Question 3: What is the technical name for the function responsible for most of the total runtime?

Quick Recap: Always profile before optimizing. cProfile shows where time is spent. Fix the biggest bottleneck first. After refactoring, run your unit tests to confirm correctness — speed and correctness must both be true.

Chapter Summary

Glossary of Key Terms

Term	Definition
Bug	Any error that causes a program to behave incorrectly
Debugging	The systematic process of finding and fixing bugs
Testing	The process of verifying that code produces correct output
Syntax Error	A grammar violation that prevents Python from parsing the code
Runtime Error / Exception	An error that occurs during execution, causing a crash
Logic Error	An error where the program runs but produces incorrect output
Stack Trace	A report showing the sequence of function calls leading to an error
Unit Testing	Testing individual functions in isolation using automated test cases
Test Case	A set of inputs and expected outputs used to verify a function
Assertion	A statement that checks a condition; raises an error if false
<code>unittest</code>	Python's built-in module for structured, class-based test cases
<code>pytest</code>	An external framework for simple function-based test cases
Edge Case	An input at the extreme boundary of valid values
Breakpoint	A marker that pauses program execution at a specific line
Watch Expression	A variable or expression tracked in real time during debugging
Step Over	Execute current line; do not enter called functions
Step Into	Enter the function being called on the current line
Step Out	Finish the current function; return to the caller
Exception Handling	Code that catches runtime errors and responds gracefully
<code>try</code>	Block containing code that might raise an exception
<code>except</code>	Block that runs when a specific exception type is caught
<code>else</code>	Block that runs only when NO exception was raised in <code>try</code>
<code>finally</code>	Block that ALWAYS runs, whether or not an exception occurred
Custom Exception	A user-defined exception class for domain-specific error types
Profiling	Measuring execution time and resource usage per function
Bottleneck	The slowest function responsible for most of the total runtime

Term	Definition
Optimization	Rewriting code to be faster or use fewer resources
Refactoring	Improving code structure/performance without changing output

Quick Reference: Choosing the Right Tool

WHICH TOOL DO I USE?	
I need to...	Use...
Prove a function works	unittest or pytest
Test unusual edge cases	pytest with multiple test functions
Find where a crash happened	Read the Stack Trace (bottom up)
Pause execution and inspect	Breakpoint in VS Code / PyCharm
Track a variable's value	Watch Expression in the debug panel
Walk through code line by line	Step Over / Step Into / Step Out
Handle a potential crash	try / except / else / finally
Make error messages specific	Custom Exception classes
Find which function is slow	cProfile
Compare two function speeds	timeit
Reduce memory usage	Generator expressions, sets

Key Principles to Remember

1. **Test early, test often.** A bug found during testing costs minutes to fix. A bug found by a real user can cost millions.
2. **Red test failures are green lights.** When a test fails, that is the test doing its job perfectly — catching a bug before real users do.
3. **Every bug has a root cause.** Think like a detective. Observe → Hypothesize → Test → Analyze → Fix → Verify.
4. **finally always runs.** Use it for cleanup code (closing files, closing database connections) that must happen no matter what.
5. **Measure before optimizing.** Never guess which part is slow. Run **cProfile** first. Then fix the actual bottleneck.
6. **After refactoring, run all tests.** Making code faster means nothing if it now produces wrong answers.

Exercise Section

Multiple Choice Questions

1. Testing is essential for reliable applications because it:

- (a) Optimises code performance automatically
 - (b) Identifies and fixes bugs before users encounter them
 - (c) Reduces the size of the program file
 - (d) Makes the code compile faster
-

2. A student writes `result = divide(10, 5)` but accidentally calls `divide(10, 0)`. The program crashes. What kind of error is this?

- (a) Syntax error
 - (b) Logic error
 - (c) Runtime error
 - (d) Compilation error
-

3. A function returns `87` instead of the correct `92`. No crash occurred. What type of error is this?

- (a) Syntax error
 - (b) Runtime error
 - (c) Logic error
 - (d) Import error
-

4. Which is the correct way to check that a function raises a `ValueError` using `pytest`?

- (a) `assert ValueError == divide(10, 0)`
 - (b) `with pytest.raises(ValueError): divide(10, 0)`
 - (c) `try: divide(10, 0) except: pass`
 - (d) `assertEqual(divide(10, 0), ValueError)`
-

5. What does the `finally` block in a `try-except-finally` structure do?

- (a) Runs only if an exception was raised
- (b) Runs only if no exception was raised
- (c) Always runs, regardless of whether an exception occurred
- (d) Stops the program after handling the exception

6. Which debugger action executes a line WITHOUT entering any function called on that line?

- (a) Step Into
 - (b) Step Out
 - (c) Step Over
 - (d) Resume
-

7. cProfile shows `process_image()` has `tottime = 4.850` and all other functions total `0.150`. What should you optimize first?

- (a) The functions in the 0.150 group
 - (b) Add more print statements to `process_image()`
 - (c) Optimize `process_image()` — it is the bottleneck
 - (d) Rewrite the entire program
-

8. What is an edge case?

- (a) An error that only occurs at the beginning of execution
 - (b) An input value at the extreme boundary of valid values
 - (c) A syntax error found at the end of a file
 - (d) A type of breakpoint placed at the edge of a function
-

9. Why is `if item in my_set` faster than `if item in my_list` for large collections?

- (a) Sets are stored on a faster hard drive
 - (b) Sets use a hash table allowing $O(1)$ constant-time lookup
 - (c) Python prioritizes set operations over list operations
 - (d) Lists require sorting before checking membership
-

10. Which Python module measures and reports exactly how many seconds each function takes to execute?

- (a) `timeit`
 - (b) `memory_profiler`
 - (c) `cProfile`
 - (d) `unittest`
-

Short Questions

1. Why is testing essential for ensuring reliable applications?
 2. What are the three common types of programming errors and bugs?
 3. What is unit testing, and why is it important in programming?
 4. How do Python's `unittest` and `pytest` modules help in unit testing?
 5. What is the purpose of writing and executing test cases?
 6. How do you set breakpoints in IDEs like PyCharm or VS Code?
 7. What is the role of monitoring variable values with watch expressions during debugging?
 8. What is the step-by-step debugging process in IDEs?
 9. How does exception handling help in managing multiple exception types effectively?
 10. What tools can be used for profiling and measuring performance in Python?
-

Long Questions

1. Why is testing essential for reliable applications, and what role does it play in ensuring program stability? Discuss the Therac-25 tragedy as a real-world example.
 2. Discuss the three common types of programming errors. For each type, explain when it occurs, how Python signals it, and which debugging tool is most effective for finding it.
 3. What is unit testing, and how do Python's `unittest` and `pytest` modules support effective testing? Include a comparison of both frameworks.
 4. Explain the process of writing and executing test cases using `pytest`. Include examples of normal cases, boundary conditions, and invalid input tests for a function of your choice.
 5. How do breakpoints and watch expressions help in debugging? Describe the complete step-by-step debugging process for finding an off-by-one error.
 6. Describe how to set breakpoints in IDEs like PyCharm or VS Code, and explain the four debugger navigation actions: Step Over, Step Into, Step Out, and Resume.
 7. How does exception handling work in Python? Explain `try`, `except`, `else`, and `finally` with a complete code example and an execution trace for both an exception occurring and not occurring.
 8. How do you handle multiple exception types effectively in Python? Write a complete program that handles at least four different exception types for a student record processing function.
-

"The next time your code has a bug — and it will — you will not panic. You will think: 'Good. A clue. Let's find it.' That is what engineers do."